



PDF Download
3694715.3695952.pdf
29 March 2026
Total Citations: 32
Total Downloads: 2702



Published: 15 November 2024

Citation in BibTeX format

SOSP '24: ACM SIGOPS 30th Symposium
on Operating Systems Principles
November 4 - 6, 2024
TX, Austin, USA

Conference Sponsors:
SIGOPS

DL Latest updates: <https://dl.acm.org/doi/10.1145/3694715.3695952>

RESEARCH-ARTICLE

Verus: A Practical Foundation for Systems Verification

ANDREA LATTUADA, Max Planck Institute for Software Systems, Saarbrücken, Saarland, Germany

TRAVIS HANCE, Carnegie Mellon University, Pittsburgh, PA, United States

JAY BOSAMIYA, Microsoft Research, Redmond, WA, United States

MATTHIAS BRUN, Swiss Federal Institute of Technology, Zurich, Zurich, ZH, Switzerland

CHANHEE CHO, Carnegie Mellon University, Pittsburgh, PA, United States

HAYLEY LEBLANC, The University of Texas at Austin, Austin, TX, United States

[View all](#)

Open Access Support provided by:

Weizmann Institute of Science Israel

Max Planck Institute for Software Systems

Microsoft Research

Carnegie Mellon University

Swiss Federal Institute of Technology, Zurich

University of Wisconsin-Madison

[View all](#)



Verus: A Practical Foundation for Systems Verification

Andrea Lattuada*
MPI-SWS

Travis Hance
Carnegie Mellon University

Jay Bosamiya†
Microsoft Research

Matthias Brun
ETH Zurich

Chanhee Cho
Carnegie Mellon University

Hayley LeBlanc
University of Texas at Austin

Pranav Srinivasan
University of Michigan

Reto Achermann
University of British Columbia

Tej Chajed
University of Wisconsin-Madison

Chris Hawblitzel
Microsoft Research

Jon Howell
VMware Research

Jacob R. Lorch
Microsoft Research

Oded Padon*
Weizmann Institute of Science

Bryan Parno
Carnegie Mellon University

Abstract

Formal verification is a promising approach to eliminate bugs at compile time, before they ship. Indeed, our community has verified a wide variety of system software. However, much of this success has required heroic developer effort, relied on bespoke logics for individual domains, or sacrificed expressiveness for powerful proof automation.

Building on prior work on Verus, we aim to enable faster, cheaper verification of rich properties for realistic systems. We do so by integrating and optimizing the best choices from prior systems, tuning our design to overcome barriers encountered in those systems, and introducing novel techniques.

We evaluate Verus’s effectiveness with a wide variety of case-study systems, including distributed systems, an OS page table, a library for NUMA-aware concurrent data structure replication, a crash-safe storage system, and a concurrent memory allocator, together comprising 6.1K lines of implementation and 31K lines of proof. Verus verifies code 3–61× faster and with less effort than the state of the art.

Our results suggest that Verus offers a platform for exploring the next frontiers in system-verification research. Because Verus builds on Rust, Verus is also positioned for wider use in production by developers who have already adopted Rust in the pursuit of more robust systems.

*Work done while at VMware Research.

†Work done while at Carnegie Mellon University.

CCS Concepts: • Software and its engineering → Formal software verification.

ACM Reference Format:

Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. Verus: A Practical Foundation for Systems Verification. In *ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP '24)*, November 4–6, 2024, Austin, TX, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3694715.3695952>

1 Introduction

Society increasingly counts on the correctness, reliability, and security of system software, i.e., fundamental software infrastructure that includes file systems, operating systems, databases, memory allocators, and libraries for cryptography and distributed protocols. Such software is often inherently low-level (e.g., manipulating raw bytes, interfacing directly with devices, or operating without a garbage collector), since higher-level software depends on it for foundational abstractions, like unlimited virtual memory or reliable operation. System software must also hit stringent performance targets, since it sits on the critical path for the software above it. However, performance optimizations, especially those involving concurrency, add complexity. Unsurprisingly, these systems are notoriously difficult to get right.

Formally verifying software is a promising approach for ensuring its correctness and reliability. Indeed, our community has seen a series of successful demonstrations verifying a wide variety of system software (§5). However, much of this success has required heroic developer effort, relied on bespoke logics for individual domains (e.g., crash safety [1–3]), or sacrificed expressiveness for powerful automation [1, 4–9].

In our work, we aim to enable faster, easier verification of rich properties for realistic systems. We do so by integrating and optimizing the best choices from prior systems, tuning



This work is licensed under a Creative Commons Attribution International 4.0 License.

SOSP '24, November 4–6, 2024, Austin, TX, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1251-7/24/11

<https://doi.org/10.1145/3694715.3695952>

our design to overcome barriers they encountered, and introducing novel techniques to simplify concurrency reasoning.

We integrate our work into Verus, a tool designed to verify Rust code. Early work [10] on Verus focused on designing and formalizing techniques for generating verification queries for both safe and `unsafe` Rust code. In this work, we describe our efforts to make Verus a powerful framework for system verification, and we evaluate our new system-relevant features in systems code (see §5 for a deeper comparison). Our ambition is two-fold: first, to make Verus a foundation for exploring the next frontier in system verification research, and second, to open system verification to a broad community of developers without deep verification expertise.

Core to Verus is the acknowledgment that verifying system software entails many kinds of reasoning at many different levels of abstraction (§2), from low-level details of memory access and bit manipulation, to the high-level challenges of defining and proving that a file system is crash-safe or that a distributed system achieves consensus. Rather than tackle this complexity with a single generic solution, Verus carefully organizes proof obligations in a way that plays to the strengths of various proof automation strategies, and hence minimizes developer effort.

Across all proof levels, Verus integrates two powerful reasoning techniques. By default, it provides aggressively optimized versions of general-purpose, semi-automated proof techniques (§3.1) that have shown success in one line of verified systems [11–16]. For portions of the system that can be described in EPR [17], a restricted logic, Verus shows how to soundly integrate another line of work [7, 8, 18–22] so proofs of these portions are fully automated (§3.2).

Verus also includes custom automation for reasoning challenges at specific levels in real systems. For example, certain coding idioms like bit manipulation and nonlinear arithmetic confound prior tools; our Verus extensions add dedicated support for such idioms (§3.3). Real systems also exploit shared-memory concurrency, so we extend Verus (§3.4) to allow a developer to describe a plan for sharding state across threads, show that threads locally obey the plan, and create invariants about the global program state from the plan.

Evaluating a new general-purpose system verification framework is challenging, given that entire papers have been devoted to verifying a single system. Hence we adopt a multi-level evaluation. We start with a set of verification “millibenchmarks”: programs we verify in Verus and in five comparable tools. This helps us assess the impact of Verus’s design decisions on verification performance in examples small enough to analyze in some detail.

We then show that the benefits seen at small scale extend to larger systems by porting the specifications, code, and proofs from several prior large-scale verification efforts to Verus and comparing the developer experience in terms of verification time and developer effort. These case studies include a distributed system client and a library that creates a

linearizable NUMA-aware concurrent data structure from a black-box sequential one.

Of course, starting from correct code and successful proofs is overly optimistic, so we also evaluate similar metrics as we verify three new system components from scratch, including an OS page table, a concurrent memory allocator, and a storage system designed for production. While our results are positive,¹ the true evaluation will come from future projects using Verus to verify systems at new levels of scale and complexity. Indeed, other research groups have already used Verus to produce verified cluster-management controllers [25], a verified microkernel [26], a security module for confidential VMs [27], and to verify LLM-produced code [28].

Any verification result is limited by its TCB. Verus’s results depend on the correctness of the top-level specifications, the Verus verifier, the solvers it relies on, and the Rust compiler.

In summary, we:

- Present the systems-relevant aspects of Verus’s design (§3), unifying prior approaches in a single framework and simplifying system proofs via novel techniques.
- Show how to soundly provide the proof-free automation achieved by prior work [7, 8, 18–22] without sacrificing expressiveness or developer freedom.
- Describe VerusSync, Verus’s new approach to allowing the developer to reason about concurrent execution *at the level of the application*, rather than the low-level mathematical objects required by prior work [3, 29, 30]
- Evaluate Verus as a system verification language through millibenchmarks and five case studies, producing over 6.1K lines of implementation and 31K lines of proof. These case studies show how Verus’s features support verifying a variety of systems for correctness and reliability. Verus also produces verification results *orders of magnitude* faster than prior automated tools.

2 Verus Overview: Verification for Systems

When constructing a complex system, developers typically tame the complexity by thinking about the system at different levels of abstraction. These range from low-level concerns like memory safety to high-level concerns like crash safety. Each level is challenging, but the ultimate challenge is ensuring that all of the levels together produce a correct system.

As detailed in Figure 1, Verus helps developers with every aspect of this reasoning. Below, we provide an overview of Verus’s key features. Given space constraints, the subsequent sections then provide more details on a selection of those features. §4 evaluates the overall impact on system verification.

Cross-Layer Proof Automation. At the scale of complexity in modern systems, encoding both the code and the abstractions needed to reason about it in a mechanized form, if done naively, can lead to huge proof obligations. Prior work (§5) tries to tame the complexity of proving correctness by

¹Verus is open source [23], as is our paper artifact and our case studies [24].

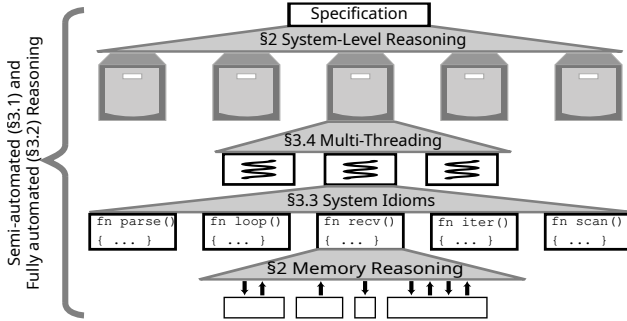


Figure 1. Verus Overview. *Verus offers powerful automated and semi-automated reasoning techniques that apply to the full system stack, as well as reasoning techniques tuned to specific stack levels.*

sacrificing either automation or expressiveness. Verus instead combines the best of both worlds, providing aggressively optimized, general-purpose, semi-automated proof techniques (§3.1) that improve on prior widely-used tools, plus full proof automation for special cases (§3.2).

Verus also includes reasoning techniques tuned for particular levels in the stack, smoothing out aspects of system verification that prior work has struggled with.

Memory Reasoning. Rather than proving the memory safety of system code via an expensive combination of manual developer effort [31–33] and/or complex encodings [11, 34, 35], Verus stands on the shoulders of the Rust [36, 37] community’s extensive work on ensuring safety through a fast, deterministic type checker. Rust’s *ownership*-based type checker enforces that only one variable has exclusive ownership of any object at a time. Rust’s growing developer population and large-scale projects in the Linux kernel [38], at Amazon [39], and at Google [40], validate that developers can use Rust’s types to verify memory safety for real systems.

Rust includes an “escape hatch” that allows developers to write explicitly labeled `unsafe` code, wherein the developer accepts the obligation to ensure the code’s safety, which challenges even experts [41]. Fortunately, Verus can verify a subset of `unsafe` Rust features [10], and it rejects usage of features outside that subset. In this work, to support our case studies (§4.2), we extended Verus’s support for raw pointers.

Hence, Verus gives programmers C-like freedom to manage and manipulate memory, and yet it automatically proves memory safety of almost all such code.

System Idioms. At the next level, Verus includes reasoning techniques (§3.3) specifically designed for thorny system idioms that the community’s experiences indicate are challenging to generically automate. These include bit manipulation [12, 14, 42] and nonlinear arithmetic [12, 43, 44].

Multi-Threading. Higher up the stack, many systems rely on concurrency for good performance, via local multi-threaded execution. Concurrent execution makes verification significantly more complex and automation challenging, since both the verifier and the developer must now consider how each thread interacts with every other thread.

Verus includes VerusSync (§3.4), a domain-specific language for reasoning about multi-threaded code directly in terms of the application’s logic, ultimately producing an abstraction of the application that encapsulates the complexity of threads, locks, and other synchronization concerns.

System-Level Reasoning. At the top level, developers often wish to prove more than the correctness of a program in isolation; they want richer properties of the program executing in a larger context. For example, a distributed system may run several instances of the program; we want the whole system to be reliable. A storage program interacts asynchronously with an external disk; we want the system to be correct even as the program crashes and restarts.

Extensive prior work [12, 13, 16, 29, 42] has demonstrated the generality and effectiveness of modeling this system-level reasoning in terms of atomic state machines [45] that capture system state and how it evolves, so Verus supports such reasoning as a special case of VerusSync.

3 Key Aspects of Verus’s Design for Systems

Here we highlight several key ideas and design decisions that enable Verus to support the verification of complex systems.

3.1 Streamlined, General-Purpose Automation

Motivation. Some of the largest verified system implementations [11–16] have relied on semi-automated *program verifiers* like VCC [11], Dafny [34], or F* [35]. Compared to *theorem provers* [31, 32], these tools are explicitly designed to verify programs, as opposed to general math theorems. While they sacrifice expressivity compared with theorem provers, they still include general-purpose reasoning techniques that allow them to be adapted to different settings. Most importantly, they provide significant automation “out of the box”, typically by converting the question of code (or proof) correctness into a query to an SMT (Satisfiability Modulo Theories) solver (e.g., Z3 [46]). The SMT solver then attempts to automatically prove the query by combining fast satisfiability solving, specialized solvers for theories like integer arithmetic, and heuristics for handling quantifiers. However, existing program verifiers struggle in the face of modern system complexity, often forcing developers to break their code into unnaturally small units and/or to tolerate long code-prove cycles.

Our Approach. Verus’s *default mode*, used in the majority of our proofs, is careful to utilize its underlying SMT solver as efficiently as possible. To illustrate, Figure 2 shows a simple example of verifiable code written in Verus. The `verus!` macro extends Rust with annotations to guide verification. For example, the **requires** and **ensures** annotations introduce pre- and postconditions that specify correctness for executable Rust functions like `pop`. These specs refer to auxiliary `spec` functions, like the `view` function that abstracts a concrete `LinkedList` implementation as a mathematical `Seq`. Internally, Verus employs standard Floyd-Hoare logic [47, 48] to convert proof obligations, such as `pop`’s

```

1 verus! {
2   pub struct LinkedList<V> {
3     head: Option<Box<Node<V>>>,
4   }
5
6   impl<V: VerusClone> LinkedList<V> {
7     pub spec fn view(self) -> Seq<V> ... { ... }
8     ...
9     pub fn pop(&mut self) -> (res: V)
10    {
11      requires old(self).view().len() > 0,
12      ensures res == old(self).view()[0] &&
13        self.view() == old(self).view().skip(1),
14    {
15      let h = self.head.take().unwrap();
16      self.head = h.next;
17      assert(self.view() == old(self).view().skip(1));
18      h.v
19    }
20  }
21 } // verus!

```

Figure 2. Verus Example. *pop* removes the first element of a linked list and returns it. Its **requires** says that *pop* can only be called if the list is non-empty, and the **ensures** says that the returned result is the list’s first value, and that it is removed from the list.

postcondition into verification conditions to send to an SMT solver (Z3 [46]), which automatically determines whether the formula holds for all possible function inputs.

The approach of converting proof obligations to verification conditions is standard in many verification frameworks, but the details of language design and implementation have a big effect on the efficiency of the SMT queries. We summarize some of Verus’s key optimizations below. The net effect is that Verus sends queries that are simpler and smaller by orders of magnitude to the SMT solver, which translates to significantly faster verification (Figure 9), so for a given verification problem, Verus can provide a more efficient code-prove cycle. Or conversely, given the same time budget as an existing tool, Verus can tackle larger verification problems.

- Unlike specification functions in Dafny [34] and F* [35], Verus spec functions are pure, total mathematical functions with no preconditions or (direct) access to the heap [10]. Thus they can be directly encoded as SMT functions without the additional baggage of prior tools [34, 35]. As prior work explains, Rust’s type system means that explicit heap-based reasoning is rarely needed, and when it is needed, it can be done through *ghost memory permissions* [10].
- Verus soundly isolates reasoning about memory, bit vectors, and nonlinear arithmetic from the main SMT queries, avoiding complex interactions between different kinds of reasoning in the SMT solver and speeding up the SMT queries. We discuss some of these ideas in §3.3.
- When verifying each module in a given project, Verus aggressively prunes the context sent to the SMT solver to eliminate imported but unreachable definitions.
- To avoid excess instantiations of quantifiers ($\forall$ and $\exists$) in the SMT solver, Verus treats quantifiers more conservatively than tools like Dafny [34] and F* [35] (see below).

Quantifiers and SMT solving. Typical systems verification projects use $\forall$ and $\exists$ quantifiers, for example to specify all

elements of an array or all members of a table. Unfortunately, SMT solving with quantifiers is undecidable in general, so SMT solvers use heuristics to decide how to instantiate $\forall$ quantifiers and how to prove $\exists$ quantifiers. Most large verification projects based on SMT solvers use an SMT heuristic based on “triggers” [49], in which the SMT solver pattern matches on expressions appearing in the formulas to determine how to instantiate quantifiers, for example, instantiating $(\forall |x: \text{u64}| \dots f(x) \dots)$ with $x = 3$ if the expression $f(3)$ arises during the proof search. Here, the expression $f(x)$ is called the *trigger* (or *pattern*) for the quantifier.

Verification tools, including Verus, automatically select appropriate expressions as triggers. However, the choice of triggers has a large impact on verification performance. Prior tools, like Dafny, default to selecting broad triggers that match many expressions, leading to many quantifier instantiations; in principle this creates more opportunities for the solver to automatically complete a proof, but in practice, for large systems projects, too many instantiations slow the SMT solver to the point where it times out and fails to complete the proof.

Therefore, Verus uses a more cautious policy that selects as few triggers as possible. If Verus is uncertain about the best trigger, Verus encourages the user to review the selected triggers and to consider overriding Verus’s default selection with the user’s choice. The result is more initial effort for users writing small projects, but better scalability to large systems projects.

3.2 Selective Use of EPR for Full Automation

Motivation. Several prior works [1, 5–8] explicitly restrict the expressivity of system properties and implementations in order to obtain powerful proof automation. The developer need only provide a little proof support, leading to remarkably low proof-to-code ratios, e.g., Hyperkernel reports 0.03:1 [4]. The Ivy verification tool [7, 8] — built on EPR (effectively propositional) logic [17] — demonstrates that despite reduced expressivity, diverse system designs and implementations can be encoded [9, 18, 50], yielding fully automated proofs and even enabling invariant inference [19–22, 51].

While EPR leads to impressive proof automation, some system components can be difficult to capture in EPR. EPR’s first-order logic admits only Boolean operators, quantifiers, and uninterpreted functions. For example, EPR can express the property that a node sends at most one message per epoch as $\forall m_1, m_2. \text{sender}(m_1) = \text{sender}(m_2) \wedge \text{epoch}(m_1) = \text{epoch}(m_2) \rightarrow m_1 = m_2$. EPR cannot directly express many common concepts, such as sequences and natural numbers; e.g., $\text{epoch}(m_2) = 1 + \text{epoch}(m_1)$ is disallowed. Instead, natural numbers are typically abstracted as a totally ordered set without arithmetic operations. EPR further requires that functions and quantifier-alternations are *acyclic* [18]; e.g., the sender function above that maps messages to nodes precludes both a function from nodes to messages and a property like $\forall n, e. e < e_{\max} \rightarrow \exists m. \text{sender}(m) = n \wedge \text{epoch}(m) = e$.

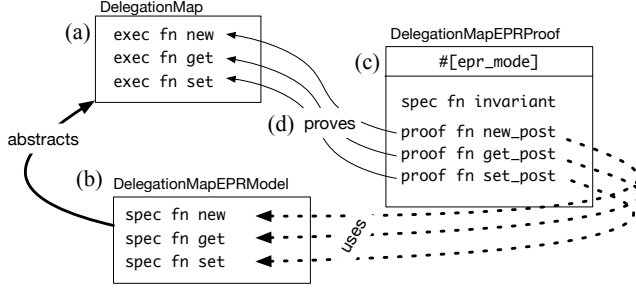


Figure 3. Verifying the IronKV Delegation Map via EPR Mode. The concrete implementation module (a) is abstracted to EPR (b) to enable a fully-automated proof (c) of its invariants, which are used (d) to prove the implementation’s correctness.

In Ivy in order to obtain the benefits of EPR proof automation the *entire* system needs to be expressed in EPR, including elements for which EPR is awkward. In contrast, using a general-purpose tool (like Dafny or F*) avoids this awkwardness but does not offer the same level of automation, even for parts of the proof where better automation may be possible with EPR.

Our Approach. In Verus we obtain the best of both worlds by soundly integrating EPR proofs with Verus’s general-purpose semi-automation; we use the latter for most parts of the system, and apply EPR to those proofs it best fits. The main challenge in enabling seamless integration of EPR and default-mode (trigger-based) automation is structuring proofs to separate the EPR and non-EPR parts, and then composing them without adding trusted components. Overcoming this challenge is enabled by Verus’s clean SMT encoding, its constructs for partitioning proofs, and its ability to control automation scoping.

To use EPR in Verus, a developer (a) defines a protocol or a data structure without restrictions, (b) abstracts the protocol or data structure into EPR, (c) uses Verus’s integrated EPR solver for automatic, predictable proofs over the abstraction, and (d) exports those proof results to the original definition where they support postconditions and lemmas.

The connection between (a) and (b) is checked using Verus’s default-mode automation, and in our experience is easily discharged. Step (c) typically involves a set of complex invariants that would have required significant manual proof in default mode, but which Verus’s *EPR mode* resolves automatically. The connection (d) back to the original proof obligations for (a) is again easily checked in default mode.

We illustrate this process on the *delegation map* data structure from one of our case studies—the Verus port of IronKV (§4.2.1). Originally written in Dafny for IronFleet [13], IronKV is a distributed key-value store where each node maintains a delegation map to associate each possible key to the host responsible for it. For efficiency, the domain is stored as a compact list of pivots that represent the key ranges. The delegation map supports three operations:

`new` creates a map with all keys mapped to one host;

`set` maps a range of keys to a new host;
`get` retrieves the host responsible for a given key.

Figure 3 illustrates the components of the delegation map and its EPR proof. First, (a) our `DelegationMap` data-structure implements the `new`, `get`, and `set` operations in regular Rust code with pre- and post-conditions expressed in an unrestricted manner, similar to the Dafny original. Our (b) `DelegationMapEPRModel` abstracts the data structure and its operations into EPR. We then (c) write an inductive invariant in EPR—`DelegationMapEPRProof`—to automatically prove an EPR version of the postconditions of each operation. Finally, we (d) use these proofs to discharge the implementation’s proof obligations for `new`, `get`, and `set`.

The implementation of the delegation map uses natural numbers for the keys. To express the proof in EPR, we abstract the keys as a totally ordered set. Verus trivially proves the soundness of this abstraction, allowing the implementation to invoke the EPR results.

The efficient list-of-pivots implementation leads to many tricky corner cases, resulting in an extensive proof in the original Dafny version. Our initial default-mode Verus proof was also extensive; e.g., the `set` operation required ~ 300 lines of proof (mostly case splitting and assertions that instantiate quantifiers) and took several days of developer effort. However, by abstracting the keys as a totally ordered set, the proof can be expressed in Verus’s EPR mode, yielding much greater proof automation. The developer still writes an inductive invariant, but the invariant is checked completely automatically, similar to Ivy [7]. However, unlike in Ivy, the implementation’s pre- and post-conditions are expressed outside of EPR, seamlessly integrating with the rest of IronKV.

At a technical level, the Verus developer marks individual modules with the `#[epr_mode]` attribute, instructing Verus to check that the module’s proof obligations are all within EPR. Verus confirms that structs have private fields so they act as uninterpreted types. It also checks acyclicity of the quantifier-alternation graph [18]. Finally, Verus enables Z3’s *model-based quantifier instantiation*, a complete (and often fast) decision procedure for EPR. Due to technicalities of Verus’s SMT encoding of polymorphic types, `epr_mode` queries are not strictly in EPR, but we have not observed any (correct) proofs that fail to verify automatically.

Our integration of EPR reasoning with a general-purpose program verifier is enabled in part by Verus’s emphasis on query economy (§3.1). Most program verifiers [11, 34, 35] produce complex SMT queries even for simple programs, immediately pushing them beyond EPR. In contrast, `epr_mode`’s SMT encoding is quite close to Verus’s default mode.

Summarizing, for modules that fit in EPR, Verus provides full automation, which prior work shows reduces developer effort by eliminating hundreds of lines of difficult proofs. Since Verus soundly embeds EPR in a general-purpose verifier, developing in a restricted logic is no longer an all-or-nothing

choice; the developer can blend proof styles as needed. We hope that the ability to seamlessly blend automation styles will allow researchers to explore new strategies for verifying large systems.

3.3 Custom Proof Automation for System Idioms

Motivation. To gain performance, systems employ various “tricks” that are challenging to automate using generic tools.

Our Approach. Rather than feed every verification problem to a generalized SMT solver, Verus includes various (trusted) automation tools, intuitively exposed to developers. In each case, Verus checks the developer’s claim fully automatically using custom automation; the SMT encoding then simply assumes it is true.

Nonlinear Arithmetic Reasoning Systems often employ nonlinear arithmetic, i.e., formulas with terms like $w \times x$ or $\frac{y}{z}$ that combine variables using operations beyond addition and subtraction. Such terms might relate adjacent page table entries (§4.2.3) or indicate the bucket to use when allocating memory (§4.2.4). Verus automates such reasoning in two ways.

First, while nonlinear arithmetic is *generally* undecidable (meaning no algorithm can solve all such formulas), subclasses are both decidable and fast in practice. Inspired by others [52, 53], Verus supports fully automated proofs of integer ring “congruence relations”—equalities built from $+$, $-$, $\times$, $\%$, and constant exponentiation—via annotation:

```
pub proof fn subtract_mod_eq_zero(a:int, b:int, c:int)
  by(integer_ring)
  requires a % c == 0, b % c == 0,
  ensures (b - a) % c == 0 {}
```

Second, for problems outside decidable fragments, we enable the SMT solver’s nonlinear heuristics in a controlled environment where they are more likely to succeed consistently. Unlike normal Verus assertions, nonlinear assertions generate an isolated query without implicit context. In the example below, even though we know from context that $q > 2$, this fact is not automatically available in the assert and must be provided as the premise of the implication. The extra developer burden provides greater predictability.

```
fn f(q: u64, a: u64) requires q > 2 {
  assert(q > 2 ==> (a * a + 1) * q >= (a * a + 1) * 2)
  by (nonlinear_arith);
}
```

Bit-vector Reasoning Low-level bit manipulation is common in systems code, whether to optimize a logarithm estimate via the *count-leading-zeroes* intrinsic or to manipulate flags or bitmaps. Unfortunately, even tools that employ automated solvers often struggle with such reasoning in the context of complex system definitions. In particular, when the solver is asked to reason simultaneously about integers for specification purposes and bit vectors for optimizations, it often fails or, worse, produces fragile proofs that succeed or fail based on minor code perturbations [44].

Hence, multiple system verification projects have developed elaborate encoding styles and proof libraries to manually guide the theorem prover [12, 14, 42]. This eliminates

instability, but it leads to a significant developer burden, e.g., requiring tens of proof lines for a simple bit mask [54].

Verus instead carefully separates integer and bit-vector reasoning. Like most tools, Verus maps Rust’s integer types (e.g., `u64`) to SMT integers for interoperability with mathematical specifications; it requires the developer to prove the absence of overflow to ensure this is sound.

However, Verus also provides an annotation-invoked mode that automatically interprets Rust’s integers as SMT bit vectors when generating an SMT query, eliminating the manual work required by prior projects [12, 14, 42]. Hence, a Verus developer proves bit-manipulation properties by writing, e.g.,

```
assert(x & 511 == x % 512) by (bit_vector);
```

Inside the assertion, `x` is a bit vector, while outside it is treated as an integer, allowing it to integrate with mathematical specifications and reliable SMT-based automation for linear arithmetic. The result is that for the cost of a small annotation, the Verus developer benefits from stable, automated proofs.

Proof by Computation Some proof obligations have obvious, statically computable answers. For example, in a previous project, we tried to verify an efficient implementation of the CRC-32 checksum that used a hard-coded lookup table of precomputed data resulting from complicated polynomial-division operations. Proving that the table resulted from this computation required an excruciating number of proof annotations to guide the solver. We eventually gave up and changed our approach to avoid the hard-coded table.

In Verus, a developer can ask that a proof be discharged by computation [55, 56]: A built-in symbolic interpreter simplifies the expression and sends any remainder to SMT.

Macro-based User-Defined Extensibility Any non-trivial systems project eventually surfaces a repetitive pattern of tedious proof; e.g., proofs that different datatypes can be unambiguously marshalled into byte arrays (§4.2.1). Just as Rust developers exploit macros to reduce repetitive code, Verus developers can do the same to reduce repetitive proofs.

3.4 Automated Reasoning for Multi-Threading

Motivation. No framework for systems verification could be complete without a story for multi-threading. Shared-memory concurrency makes verification significantly more complex, requiring reasoning about all possible thread interactions. Rust’s memory safety guarantees extend to multi-threaded code, as captured by its goal of “fearless concurrency.” However, it remains far from obvious how to cleanly prove full correctness of concurrent systems.

Our Approach. Verus combines two big ideas.

Idea I: Resource Algebras The first idea is something called a *resource algebra* [57], a proven concept from concurrent separation logic [58]. A resource algebra is a set of rules for creating *ghost resources*, which can help maintain invariants between objects owned by different threads.

```

1 // ----- VerusSync -----
2 fields {
3   #[sharding(variable)] pub a: int,
4   #[sharding(variable)] pub b: int,
5 }
6
7 init!{ initialize(val: int) {
8   init a = val; init b = val;
9 }}
10
11 transition!{ update(val: int) {
12   update a = val;
13   update b = val;
14 }}
15
16 property!{ agreement() { assert pre.a == pre.b; } }
17
18 #[invariant]
19 pub spec fn agreement_invariant(&self) -> bool {
20   self.a == self.b
21 }
22
23 // ----- Agreement Protocol -----
24 type Agree;
25
26 fn initialize(val: int) -> (pair: (Agree, Agree))
27   ensures pair.0.id() == pair.1.id() &&
28   pair.0.value() == val && pair.1.value() == val;
29
30 fn update(a:&mut Agree, b:&mut Agree, new_value:int)
31   requires old(a).id() == old(b).id(),
32   ensures a.id() == b.id() &&
33   a.value() == new_value && b.value() == new_value;
34
35 fn agreement(a: &Agree, b: &Agree)
36   requires a.id() == b.id(),
37   ensures a.value() == b.value();

```

Figure 4. A Simple VerusSync Example of Keeping Two Values In Agreement. Given the VerusSync code (top), Verus automatically generates relevant proof obligations, such as the fact that `agreement_invariant` is inductive; in this case Verus’s SMT solver dispatches them without additional proof work. Once these proofs succeed, Verus generates the relevant resource, the update operation (`update`), and the proof result (`agreement`). The result is the agreement protocol interface (bottom part).

As a simplified example, consider a program with two objects and a developer who wishes to maintain the invariant that the objects both store the same value. Without verification, the developer could informally enforce this invariant by writing code that follows a *protocol* in which an update can only be performed when both objects are held by the same thread (as formalized in Figure 4 (bottom)). The protocol intuitively enforces the invariant, but how can we prove it? During times when the objects are owned by disjoint threads, “who” maintains the invariant? The resource algebra solves this problem, giving us a way to determine sound “update” operations for a given invariant.

Rust’s ownership types are a natural representation for ownership of resource algebra resources. In fact, this idea has been employed before; IronSync [29] shows how to do it with the ownership type system of Linear Dafny [16], and this in turn inspired Verus’s ownership-based ghost types [10]. However, the monoid-based mathematical formalism behind resource algebras is quite technical; it takes considerable expertise to interpret monoids in the context of a concurrent system. Thus,

deploying this idea in practical systems verification requires a second big idea to streamline the process.

Idea II: A Specification Language for Transitions To enable a practical, streamlined process for constructing the complex ghost state needed for sophisticated concurrent algorithms, we introduce a novel framework called VerusSync. To design VerusSync, we first observe that a resource algebra update, which exchanges one set of resources for another, is fundamentally a *transition*. We posit that state transitions are an intuitive basis for reasoning about a system. However, in the canonical resource algebra formulation, a developer derives these transitions by a set of rules based on the compositional monoid structure of a resource algebra, which is less intuitive as a basis for system reasoning.

In VerusSync, therefore, we make transitions the central object for reasoning. In a VerusSync system, the developer specifies transitions up front, and thus VerusSync has special syntax for transitions, based on the elegant state-transition syntax in Ivy [7, 8], which has been widely used to verify concurrent distributed systems, a close relative of concurrent multi-threaded systems. As with Ivy, VerusSync transitions are described by enabling conditions (specifying when a transition is allowed) and state updates (see Figure 4, top). The unique part of VerusSync is its special “sharded” update commands, illustrated below.

In showing that a VerusSync system is well-formed, the key proof obligations are *safety conditions*, which the developer proves by supplying an inductive invariant. Supplying inductive invariants may still be nontrivial, but it is a widely understood process that allows developers to draw from experience with loop invariants and distributed system invariants. Because of this, our experience has shown us that resources are easier to specify and prove correct this way than via monoidal composition. However, VerusSync still appeals to the same underlying theory behind resource algebras; our metatheory shows that a well-formed VerusSync system (i.e., one satisfying the inductiveness conditions) always corresponds to a resource algebra with the necessary properties.

VerusSync In Action. In VerusSync, the developer first constructs a state comprised of fields, each labeled with a *sharding strategy* defining how the field relates to ghost *shards* manipulated in the code. For example, a **variable** field is represented by a single shard; a **map** field is represented by one shard for every key-value entry in the map.²

Next, the developer defines the protocol as VerusSync transitions. Each “update” operation has one meaning for the aggregate state and a corresponding meaning as an operation on shards. For example, the **add** keyword adds a key-value pair to a state’s **map** field, and correspondingly creates a shard containing the pair. Similarly, **remove** means “remove a key-value pair” and “consume a shard”.

² The sharding strategies together define the monoid in the underlying resource algebra, a formality VerusSync hides from developers.

NR Queue. At the core of our NR case study (§4.2.2) is a ring buffer that tracks three pieces of state in shared memory:

- a buffer for storing message entries,
- a *tail pointer* where the next message goes, and
- a per-thread *head pointer*, indicating where each thread should read the next message.

There is a complex, multi-step protocol for reading and processing messages from the queue. This protocol is implemented by a thread called the *executor thread*. As the queue operates, no one thread ever simultaneously owns all the pieces of state above, as they are accessed and updated concurrently. Even so, we maintain complex invariants, e.g., relating an executor thread’s internal state to its head pointers, or relating the head and tail pointers to non-empty buffer entries.

We model this protocol as a VerusSync ghost resource with fields representing the pieces of state, and we distribute the resulting shards across the threads. The ring buffer’s fields are sharded with different strategies.

- The `tail` field marks the next empty space. Its value is represented by a single shard that a thread must own to read or modify the field. The ghost shard is associated with a particular physical memory word accessed atomically, here via compare-and-swap.
- The `buffer_size` field is constant: all threads agree on (can “read”) its value; it is permanently read-shared.
- The `local_versions` field contains per-thread head-pointers into the queue. It is represented by a map where each entry is an ownable shard, each associated with a different atomically-accessed memory word.
- The `executor` field describes the intermediate state of each executor thread within its multi-step protocol.

In NR, an executor thread pops operations off the queue and processes them. The thread selects a range to read, reads each buffer entry in that range, and finally updates the atomic field corresponding to an entry of `last_version` to point to the end of this range. Figure 5 shows this final step of the protocol, as expressed in VerusSync. Verus then generates appropriate owned ghost shards, which the programmer manipulates in their executable code to prove that the code correctly follows the protocol. Figure 6 shows one way to arrange an executable data structure to maintain the relationship between the concrete data and the corresponding ghost shards. The style is similar to that in IronSync, which provides an in-depth example [29, §3.3.2].

The VerusSync approach lets Verus determine, *syntactically*, that the transition only affects two shards. This then allows the developer to perform the operation when they have ownership of those two shards *without* ownership of any of the many other shards in the system. Finally, because VerusSync allows us to easily interpret this system as a transition system, we can use traditional state-machine techniques to prove global properties of the ring buffer, most notably that the threads’ accesses to the buffer entries are well-formed.

```

1 enum ReaderState {
2   // We have identified the start point of the range
3   // to be processed, but not yet the endpoint.
4   Starting { start: LogIdx },
5   // We are processing the range 'start..end', and
6   // the next entry to be read is 'cur'.
7   Range { start: LogIdx, end: LogIdx, cur: LogIdx },
8   // ...
9 }
10
11 enum ExecutorState {
12   Idle,
13   Reading(ReaderState),
14
15   // Other states for phases unrelated to reading
16   AdvancingHead { /* ... */ },
17   AdvancingTail { /* ... */ },
18   Appending { /* ... */ },
19 }
20
21 // ----- VerusSync -----
22 fields {
23   #[sharding(variable)] pub tail: LogIdx,
24   #[sharding(constant)] pub buffer_size: LogIdx,
25   #[sharding(map)]
26   pub local_versions: Map<NodeId, LogIdx>,
27   #[sharding(map)]
28   pub executor: Map<NodeId, ExecutorState>,
29   /* ... */
30 }
31
32 reader_finish(
33   node_id: NodeId, start: nat, end: nat, cur: nat) {
34   // Advance node's state from `Reading` to `Idle`.
35   // The 'require' condition indicates a precondition
36   // that we be at the end of the range we originally
37   // selected for this executor phase.
38   require(cur == end);
39   remove executor -= [ node_id =>
40     ExecutorState::Reading(
41       ReaderState::Range(start, end, cur)) ];
42   add executor += [ node_id => ExecutorState::Idle ];
43
44   // Advance the node's current version to `end`.
45   // The transition has no precondition on the pre-
46   // state value of the version, but CyclicBuffer's
47   // invariants imply that it increases in this step.
48   remove local_versions -= [ node_id => let _ ];
49   add local_versions += [ node_id => end ];
50 }

```

Figure 5. A More Advanced VerusSync Example. An executor thread completing an update takes this `reader_finish` transition, which is only possible if the thread owns two shards needed for `remove`: one showing the executor is working on `Reading` a range that finishes at `end`, and another that proves it owns the right to mutate the `node_id` version entry. The transition replaces those shards (via `add`) with one that notes the executor is now `Idle` and another that stores `end` into the `node_id` version entry.

4 Evaluation

We evaluate Verus as a system verification language across two key dimensions. First, is its proof automation sufficiently powerful and fast for complex system verification? Second, is it expressive enough to specify important system properties and give developers the freedom to write high-performance code that satisfies them?

An ideal evaluation would build K large systems in N different frameworks; however, each such system might warrant an entire paper. Instead, we adopt a pragmatic, multi-scale evaluation strategy. First, we conduct “millibenchmarks” that compare Verus against many verification frameworks on small

```

1 pub struct NrLog {
2   // Named after the AtomicU64 type in Rust's
3   // standard library, Verus's standard library
4   // provides an AtomicU64 type that supports
5   // ghost shards. The vector holds one per replica.
6   pub local_versions:
7     Vec<AtomicU64<LocalVersionShard, ...>>,
8   // ...
9 }
10
11 invariant on local_versions
12   forall |i: int|
13     where (0 <= i < self.local_versions.len())
14     specifically (self.local_versions[i])
15     is (val: u64, shard: LocalVersionShard)
16 {
17   shard.key == i
18   && shard.value == v
19   && 0 <= v <= MAX_IDX
20   && ...
21 }

```

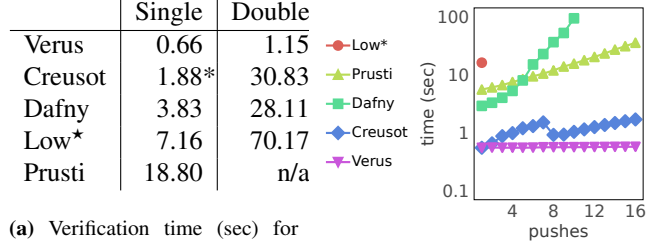
Figure 6. Linking Executable Code to VerusSync Transitions. Through a combination of trusted Verus primitives and verified utility code from Verus’s “standard library,” we can enable multiple, concurrently-running threads to manipulate the *LocalVersionShard* by associating it with an atomic memory cell that can be accessed in a safe, concurrent manner. To streamline this process, Verus provides the invariant on ... is ... syntax (shown here) to easily write a predicate that connects the ghost shard with the physical value of the atomic cell. When the code atomically updates the physical value, the code also invokes a VerusSync API to update the corresponding ghost shard (e.g., by invoking the *reader_finish* step), and maintain the invariant.

but representative system-relevant tasks. Second, macrobenchmarks show that these benefits translate to benefits at scale. In total, we verify over 6.1K source lines of Rust code, entailing over 31K source lines of proof. Except where noted, the experiments below are conducted on an AWS EC2 m5.8xlarge instance (Intel Xeon Platinum 8175M @ 2.50GHz) with 16 cores (with multithreading disabled) and 128GiB of memory.

4.1 Millibenchmark Evaluation

We begin by designing a series of “millibenchmarks”, each large enough to capture an important system verification task, but small enough that we can implement them in multiple verification frameworks and analyze them in some detail. We measure the wall-clock time to verify each version. To reduce the risk that we use another verification framework naively, we draw our benchmarks from those provided by the frameworks themselves, so we presume they are reasonably optimized. Where we change examples or port new ones to existing frameworks, we have confirmed with the framework’s designers that our artifacts are reasonably idiomatic.

4.1.1 Frameworks We focus on comparison to verification frameworks that (a) have been used to verify complex properties of large-scale systems, and (b) offer a large degree of automation “out of the box”. This includes **Dafny** [34], which has verified cryptographic code [53, 59], application stacks [29], distributed systems [13], storage systems [16], and production-scale multi-threaded systems [29]; and **F*** [35] (specifically the Low* subset [60]), which has verified ~43K



(a) Verification time (sec) for the singly linked list and doubly linked list millibenchmarks. Prusti cannot express cyclic pointers. (b) Verification time when varying the number of pushes to four singly linked lists. Note the log-scale y-axis.

Figure 7. Millibenchmarks. Median of 20 single-threaded runs. *This measures the non-interactive time for Creusot to produce a partial result; completing the proof requires manual intervention.

lines of C and assembly code in a cryptographic provider [14], the TLS 1.3 [61] and QUIC [62] record layers, and the Signal messaging protocol [63]. We also include **Ivy** [7, 8], which has verified distributed system protocols [9, 18–22], as a representative of tools trading expressivity for proof automation.

To confirm that Verus’s benefits arise from design decisions beyond simply building on Rust, we include two state-of-the-art automated Rust verification frameworks, **Prusti** [64] and **Creusot** [65], even though they do not verify concurrent code and have not yet been applied to large-scale systems projects.

4.1.2 Millibenchmarks We start by defining three general-purpose millibenchmarks on basic data structures. Even these simple examples fall outside the restricted logics of Ivy [8] or Serval [5], so we add a benchmark that fits in Ivy’s EPR.

Singly linked list. To evaluate verification efficiency on a small task, we verify that a singly linked list implements an abstract sequence. The verified API is consistent across the verification tools. The list supports pushing at the head, popping at the tail, indexing, and iteration.

Doubly linked list. To evaluate how verification performance scales with complexity, we implement a doubly linked list and prove it implements an abstract sequence, which requires *unsafe* Rust because of its cyclic pointers. This list supports pushing and popping at both ends and iteration.

Memory reasoning. Reasoning about memory updates at scale is a perennial challenge for system verification. Hence, using the verified lists, we evaluate the cost of memory reasoning by repeatedly updating four instances of the list within the same function and then asserting basic facts about the elements of the lists. This requires the verifiers to determine whether an update to one list might affect another.

Distributed lock. To evaluate distributed protocols, we port a distributed lock to Verus and prove mutual exclusion in two ways: in default-mode following the Dafny proof [13], and using Verus’s EPR mode, similar to the Ivy proof [7].

4.1.3 Millibenchmark Results Linked lists. Figure 7a shows the verification time for our two linked list examples. For the singly linked list, we see that the other frameworks take 3–28× longer than Verus, while for the more complex

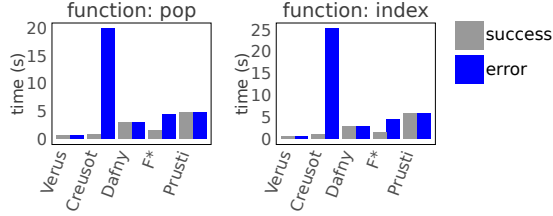


Figure 8. Error Results. Median of 20 runs. Tools use 8 threads.

doubly linked list, they take 24–61× longer. A major cause is Verus’s emphasis on concise SMT queries (§3.1, Figure 9).

Successful verification times are the easiest apples-to-apples comparison, but in reality, developers most often wait on tools for failure feedback. To capture this, we “break” each singly-linked-list proof twice, once by removing a precondition in `pop` and once in `index`, and measure the time for the tool to report an error. Figure 8 shows that Verus, Dafny, and Prusti pinpoint failures as quickly as they report success. Low* degenerates from one second to four. Creusot’s approach degenerates from one second to twenty.

Memory reasoning. Figure 7b shows how the verification frameworks handle increasing numbers of memory modifications to four singly linked lists. Dafny and Low* must perform complex aliasing reasoning, and hence Dafny’s verification time grows dramatically as memory modifications increase. Low* struggles even more, failing to return after more than one push. Prior Rust-based tools perform better, but they still grow super-linearly,³ whereas Verus remains linear (with a slope of ~1.6ms/push) across the entire benchmark. The results for doubly linked lists are similar, with Verus remaining linear with a slope of ~1.8ms/push.

Distributed lock. The safety proof uses an inductive invariant maintained across protocol steps. The default-mode proof of inductiveness is around ~25 lines. When abstracted into EPR, the proof is automatic, but creating and using the abstraction required ~100 lines of (straightforward) boilerplate. While this example demonstrates that Verus’s EPR mode applies to protocols as well as data structures, the excessive boilerplate suggests that (a) EPR benefits complex examples, like the Delegation Map in §3.2, more than simple ones, and (b) Verus needs to automate the boilerplate.

4.2 Macrobenchmarks

To show that the benefits of Verus identified by the millibenchmarks translate into benefits at scale, we assembled a suite of macrobenchmarks. First, to compare with other frameworks at scale, we port two large verified systems to Verus (§4.2.1, §4.2.2) and compare each to its original.

Second, since developers rarely start from completed code and proofs, we examine the tool’s behavior in its primary mode, when verification *fails*. To explore fresh development,

³Creusot “races” several solvers in parallel; the dip in the graph shows when a different solver starts to “win”.

System → Verifier	Line Count			P/C Ratio	Time (s)		SMT (MB)
	trusted	proof	code		1 core	8 cores	
IronKV							
→ Verus	1613	4509	1533	2.9	41	18	17
→ Dafny	1205	8070	1923	4.2	445	201	352
NR							
→ Verus	369	5237	736	7.1	17	9	22
→ L.Dafny	104	7828	730	10.7	1089	228	2063
Page table	1117	5329	400	13.3	63	34	37
Mimalloc	282	13703	3178	4.3	262	55	152
P. log	1284	2913	739	3.9	12	6	9
Verus total	4692	31231	6119	5.1			

Figure 9. Macrobenchmark Statistics. Verification performance and proof overhead of each benchmark, including the original verified systems we ported to Verus. Times are seconds to successfully verify the entire project. SMT is the total size of queries sent to Z3.

we report on the experience of writing and verifying three additional systems from scratch in Verus (§4.2.3–§4.2.5).

4.2.1 Porting IronKV from IronFleet [13] IronFleet [13], originally developed in Dafny, allows developers to prove that a distributed system’s implementation is both safe and live. This requires proofs about both the implementation that runs on each host, and the protocol the hosts use to achieve the system’s high-level properties.

Verification Target. We port the host implementation of IronFleet’s IronKV, which dynamically shards a key-value store across a set of nodes. We skip the protocol level, since Dafny and Verus share similar mathematical modeling tools and are likely to admit very similar proofs. We translate the protocol-level host description to Verus as the goal spec, implement the host in Rust, and prove it matches this spec.

Porting Experience. To support comparisons, our port preserves IronKV’s algorithmic decisions, but, where highlighted below, our design exploits Verus-enabled improvements.

Basic Improvements We encountered multiple places where IronFleet split a simple task across many functions, presumably to keep verification times manageable. For example, IronFleet’s `MaybeAckPacketImpl` accepts a message and looks up its sequence number in a tombstone table to decide whether to ack it. In IronFleet, this simple task is split into three functions across 37 lines. Our port inlined those three functions into a single 30-line Verus function that verifies faster than any of the three original Dafny functions.

In another place, the IronFleet code dealt with the painfulness of reasoning about fine-grained mutation by replacing an entire data structure with a modified version, leading to inefficient (and confusing) code. The Verus port simply uses an `&mut` reference, which expresses the original intent directly and avoids a performance penalty.

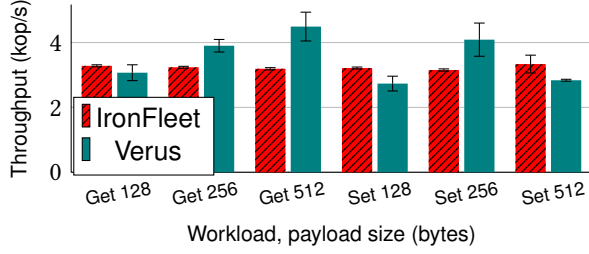


Figure 10. IronKV Performance. The Verus version performs comparably to the IronFleet original. Each bar shows the mean of 100 trials; error bars show 95% confidence intervals.

Parsing and Marshalling In IronFleet’s IronKV there is a generic marshalling library for basic types: arrays, tuples, tagged unions, 64-bit integers, and byte-arrays. The developers mapped Dafny datatypes to and from these basic types, manually constructing tedious boilerplate code and proofs.

We instead wrote our own marshalling library that eliminates this tedium via user-defined macros (§3.3) and provides a more ergonomic interface using traits. Our `Marshallable` trait (including a marshaller, parser, and relevant lemmas), provides a consistent interface, improving on the original which relies on naming convention. Primitives (like `u64`) and repetition (`Vec<T>`) implement this trait with hand-written proofs. Arbitrary `structs` and `enums` use macros that automatically derive implementation and proofs, eliminating the repetitive manual proofs prevalent in the Dafny original.

EPR Delegation Map Proof Using Verus’s EPR mode drastically simplified the proof of this data structure (§3.2).

Evaluation. Figure 9 shows that the Verus port saves both code and proof, improving the proof-to-code ratio. The *trusted* column is noisy because of how we reinterpreted the original protocol layer as our spec. The authors that completed this port reported crisp interactivity, supported by the 95% smaller query sizes and 10× faster verification.

To confirm the fidelity of our port, we benchmark both systems with the test harness from IronFleet’s repository [66]. The experiments run on Windows 11 Enterprise on a 2.4GHz Intel Core i9-10885H CPU 8-core laptop with 64GiB of RAM. We launch three server processes on separate ports, then launch the client workload generator with 10 threads and 10,000 keys for 30 seconds. We vary the workload (Get vs. Set) and the payload size (currently limited to 512 bytes by the IronFleet repository). Since our port is fairly faithful, we anticipated similar performance, which Figure 10 confirms.

Summary. Our success porting the IronKV host demonstrates that Verus at least subsumes the functionality of Dafny employed by IronFleet, and that it is largely compatible with that approach to system verification. It improves by moving heap reasoning into ownership reasoning, which eases performant implementation of mutable data structures and, by optimizing solver performance, enables coalescing tasks into more reasonably-sized functions. Rust’s macro system and Verus’s EPR support significantly reduce developer tedium.

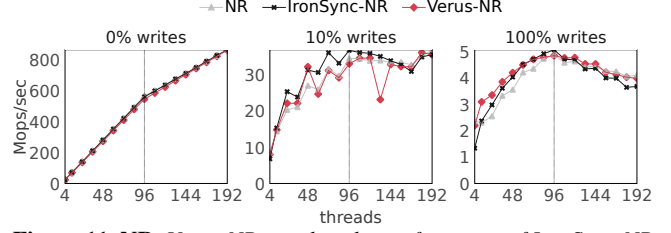


Figure 11. NR. Verus-NR matches the performance of IronSync-NR (and the unverified original) despite a much easier proof.

4.2.2 Porting NR from IronSync [29] The IronSync framework [29] enables verifying the correctness of complex shared-memory programs that employ application-specific synchronization primitives to achieve high performance. It is built on Linear Dafny [67], a variant of Dafny extended with simple Rust-inspired ownership types.

Verification Target. We ported the IronSync implementation of the Node Replication (NR) concurrency library [68]. NR converts a sequential data structure into a high-performance, concurrent version via replication and flat combining. We prove the same result as IronSync, namely that the concurrent system meets the sequential functional spec *linearizably*.

Porting Experience. Our Verus-NR implementation is more faithful to the original NR than IronSync-NR in three ways. First, both NR and Verus-NR are written in Rust; IronSync-NR was a port to Dafny. Second, Verus-NR exposes a trait-based interface similar to NR’s in order to support generic data structures, whereas IronSync does not support traits. Finally, Verus-NR admits runtime-defined counts and dynamic thread registration, whereas IronSync-NR fixes the replica and thread counts statically.

Furthermore, proofs related to concurrency are substantially simplified in Verus-NR due to the use of VerusSync (§3.4) rather than IronSync’s monoid formalism. VerusSync allows cleaner, application-level reasoning, and the simplification is reflected in our reduction in proof code.

Evaluation. Figure 9 shows that Verus-NR requires far fewer lines of proof. This is mostly due to the use of VerusSync. Verus also improves the verification time by *two orders of magnitude*. Such speed creates a qualitative advantage: Where we might verify one function at a time with a slower tool, we iterated while verifying the entire project, which provides an early warning when a change to a function contract breaks a proof elsewhere.

We compare Verus-NR’s performance with unverified NR and with IronSync-NR using the benchmark from IronSync’s artifact [69]. We run this benchmark on a four-socket Intel Xeon Platinum 8260 with 24 cores and hyper-threading enabled. The benchmark initializes NR with four replicas wrapping an x86-page-table data structure. We increase the thread count, filling up NUMA cores before utilizing hyperthreads. We measure the throughput at write ratios of 0%, 10%, and 100%. Figure 11 shows that Verus-NR’s performance matches the unverified, highly optimized original implementation.

Summary. Porting NR shows that Verus can verify state-of-the-art concurrent data structures optimized via application-specific synchronization primitives. It does so faster, more intuitively, and with less developer tedium than existing state-of-the-art concurrent verifiers.

4.2.3 New Verified System: An OS Page Table To evaluate Verus when developing a new verified system, we implement a verified page-table data structure for x86-64.

Verification Target. The map and unmap operations of a page table entail traversing and modifying a tree data structure whose nodes pack flags and addresses into 64-bit machine words. Verifying it requires specifying and reasoning about ISA-level software and hardware components.

Correctness is specified from the perspective of a user-space process on a single-processor system: reads return the most recently written value; map and unmap operations expand and restrict the virtual memory domain. The implementation employs a (trusted) hardware spec that defines how the MMU interprets page table memory to translate virtual addresses to physical.

Development Experience. We highlight two aspects of how Verus enables necessary low-level reasoning: bit-level manipulation of 64-bit words and specifying how the implementation may interact with page-table memory.

Page-table entries are bit-packed 64-bit words. To reason about them efficiently, we rely heavily on Verus’s automation for bit-vector, nonlinear, and proof-by-computation reasoning (§3.3), which we invoke 62, 39, and 11 times, respectively. They enable us to automatically discharge conditions like:⁴

```
forall |a:u64,i:u64| i < 13 && (a & mask!(13u64,29u64) ==
  0) ==> ((a | bit!(i)) & mask!(13u64,29u64) == 0)
```

which in other frameworks [34, 35] would have incurred tedious manual proof [14, 54].

The trusted spec of the MMU describes how it translates memory accesses based on the values of page table entries. This interpretation is only meaningful with respect to the physical values in the memory storing the page table. The trusted spec provides a struct that encapsulates ownership (and allocation) of the page-table memory: ownership prevents other writes to the entries, the encapsulation tracks the values of the entries as ghost state, and the MMU contract can thereby make promises about its translation. Ownership facilitates soundly specifying this hardware behavior.

Evaluation. As shown in Figure 9, our page table consists of 400 lines of executable code, which required 5329 lines of proof, resulting in a relatively high 13.3:1 proof-to-code ratio. This may be a result of this being our first large-scale development in Verus; more experience may suggest different abstractions for some proofs. A page table is also a complex OS component, so a high ratio may be inevitable. When it comes to verifying a complete OS, independent work uses

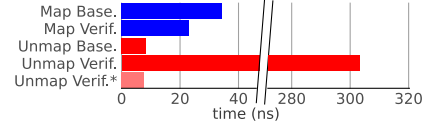


Figure 12. Page Table Run-time Performance.

Verus to verify a full microkernel [26]; the authors report a 7.5:1 proof-to-code ratio.

We compare our implementation against a recent unverified page table implementation [68] in a single-threaded setting, reporting mean latency over 100M map and unmap operations on 4K frames. Figure 12 shows our implementation matches the reference for mapping frames, but our unmap is much slower. This discrepancy is because we reclaim emptied page directories, which we confirm by benchmarking an (unverified) modification of our page table with reclamation disabled (Unmap(Verif.*) in the figure). Larger OS-level benchmarks show negligible differences, even with the cost of our unmap.

Summary. This system demonstrates Verus’s ability to specify both OS and hardware interfaces and reason about a complex low-level implementation connecting the two.

4.2.4 New Verified System: Memory Allocator As our largest, most complex new system, we develop a concurrent memory allocator.

Verification Target. Our work is based on *mimalloc* [70], which provides state-of-the-art performance. Since *mimalloc* is written in C, our Verus version translates C into Rust idioms, but preserves the overall data structures and algorithms. We prove our implementation functionally correct, meaning that every allocation returns non-aliased memory.

Development Experience. Any memory allocator system faces two significant challenges.

Address space management. The overall objective of a memory allocator is to bridge the gap between an OS memory API that supports *coarse-grained*, page-aligned allocations (e.g., the `mmap` syscall on Linux) and an allocator API supporting *arbitrary-sized* allocations (`free` and `malloc`). This requires careful accounting of the address space.

To reason about the address space, we use ghost memory permissions [10]. We write a trusted specification for `mmap` in terms of ghost permissions; these permissions can then be passed up to the client when they call `malloc`.

Cross-thread deallocations. A client may free memory on a different thread from which it was originally allocated. *Mimalloc*’s design handles this by depositing cross-thread deallocations into an *atomic free list*, a lock-free linked list accessed via atomic compare-and-swap.

To do the same, we utilize Verus’s ability to associate ghost state with atomic locations (§3.4). Specifically, we deposit ghost memory permissions from cross-thread deallocations into the atomic variable holding the linked list’s head pointer.

Both address space management and cross-thread deallocations are challenging due to the complexity of memory

⁴`mask!(x, y)` sets the bits between `x` and `y`.

Benchmark	mimalloc	Verus-mimalloc
cfrac	4.6 s.	9.7 s.
larsonN-sized	4.1 s.	12.0 s.
sh6benchN	0.14 s.	2.0 s.
xmalloc-testN	0.34 s.	0.73 s.
cache-scratch1	1.2 s.	1.2 s.
cache-scratchN	0.16 s.	0.16 s.
glibc-simple	1.2 s.	6.6 s.
glibc-thread	1.1 s.	3.6 s.

Figure 13. Mimalloc Benchmarks Supported by Verus-mimalloc. Benchmarks run on Linux on an 8-core, 3.60GHz Intel i9-9900K. The mimalloc authors label *cfrac* and *larsonN-sized* as “real world” benchmarks and the others as pathological stress tests.

ownership across threads. This complex ownership structure is implicit in the original C codebase; porting to verified Rust requires us to make the ownership structure explicit. With ghost state, this is possible (and necessary) to a degree even beyond what could be done in unverified, unsafe Rust.

Evaluation. We have implemented a subset of the features and optimizations supported by mimalloc’s design. For comparison’s sake, Verus-mimalloc has about 3.1K lines of executable code, while the original is about 10K lines of code. Our allocator supports use as a drop-in replacement for the system allocator with some limitations: it does not yet support `realloc`, aligned allocations, or allocations greater than 128KiB. It can complete 8 out of 19 benchmarks from mimalloc’s benchmark suite [71], though it does not yet reach performance parity (Figure 13). We focused our initial development on aspects highlighted in the mimalloc report [70], particularly those affecting concurrency, so we believe Verus-mimalloc is prepared to support the missing features and optimizations in the future.

Figure 9 shows a favorable proof-to-code-ratio of 4.3. Furthermore, the allocator’s user-facing specification is very succinct: between initialization, `malloc`, and `free`, it is only 37 lines. The allocator relies on OS interfaces (`mmap` and thread utilities) with 245 lines of trusted spec, bringing the total to 282. Our allocator also relies heavily on Verus’s bit-vector, nonlinear, and proof-by-computation automation (§3.3), which we invoke 78, 71, and 187 times, respectively.

Summary. Totalling over 17.2K lines (code and proof), Verus-mimalloc is the largest of our macrobenchmarks and the largest Verus project we know of. Even so, Verus verifies the entire project in just over a minute.

4.2.5 New Verified System: Persistent Log To evaluate Verus’s utility for verifying production code, we developed a persistent circular log for byte-addressable storage devices such as Optane DC Persistent Memory [72].

Verification Target. The log offers asynchronous `append` and synchronous `advance_head` operations to its storage system client, and it supports atomic appends to multiple

separate logs. The log will only overwrite space freed via `advance_head`.

Our log is designed for storing low-level metadata and data in a cloud-scale production storage system. It is integrated into a production codebase, which incorporates it via `Cargo.toml` as just another Rust crate.

Development Experience. We verify the implementation refines an abstract, infinite log; that all operations are atomic with respect to crashes; and that the log metadata is protected from corruption up to CRC. These properties are essential for persistent memory, which has a small persistence granularity and is at risk for fine-grained media errors, random bit flips, and stray writes [73]. They are also especially valuable in cloud-scale storage, where crashing and corruption bugs too rare to detect with traditional testing still turn up.

Production integration is simple: Verus erases all but Rust content for tools other than the verifier; standard `rustc` sees only executable code and readily consumes it. For the crates that the verified code depends on, such as a CRC crate, we write a specification and mark it trusted.

To simplify our proofs, our initial verified log converted each metadata structure to a byte slice before writing to persistent memory, incurring unnecessary copying in DRAM. Our latest version provides a `Serializable` trait with `spec` methods to specify the byte-level layout of metadata structures. Structures that implement this trait can be copied directly to persistent memory without the need for runtime conversion to a slice, removing conversion overhead while providing the same guarantees.

Evaluation. Verus verifies the log implementation in 12s with a proof-to-code ratio of 3.9 (Figure 9), while offering correctness, crash safety, and metadata-corruption detection.

We evaluate performance on a 128GiB Optane Persistent Memory Module device. All log updates are written directly to the device through a 4GiB memory-mapped file in Ext4-DAX [74]. We compare the latest verified log against `libpmemlog` [75] from the state-of-the-art PMDK [76], and against the original log prototype.

Figure 14 compares the append throughput of all three systems with 95% confidence intervals. Each data point is 8GiB of appends (including operations to free space so the log can wrap around). The initial version of the verified log provided low throughput on small appends due to its extra copying; the latest version eliminates this overhead and achieves comparable throughput to `libpmemlog`. The current verified log and `libpmemlog` have similar throughput even though the verified log calculates CRCs and `libpmemlog` does not, because `libpmemlog` acquires and releases a lock on each append while the verified log uses no locks.

Summary. The verified log shows that Verus can develop software that integrates naturally into production code bases. It also demonstrates that Verus supports reasoning about domain-specific properties, like crash safety, without “baking” such reasoning into Verus itself.

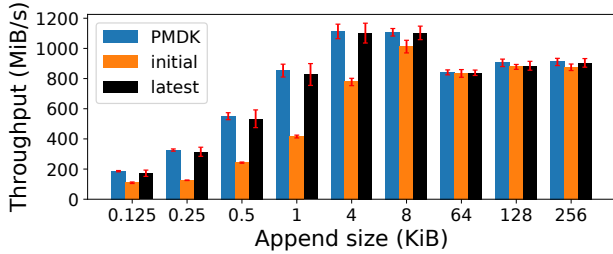


Figure 14. Verified log vs. libpmemlog. Append throughput

5 Related Work

Multiple groups [2, 3, 77–82] have employed *interactive theorem provers* like Coq [31] and Isabelle/HOL [83] to verify systems. By default, these are less automated than SMT solvers: developers manually walk the verifier through the proof by applying tactics, leading to large proof-to-code ratios (e.g., over 20 : 1 for the initial version of seL4 [84]), despite some recent domain-specific improvements [82].

In contrast, Verus continues a line of work on *program verifiers* [11, 34, 35, 64], which focus on verifying programs written in a particular language. Using solvers, these tools typically offer more automation “out of the box” (e.g., the Ironclad project [12] had a proof-to-code ratio of 5 : 1).

Verus benefits from prior work on using ownership type systems to simplify memory reasoning in systems code. For example, a prior study on Linear Dafny [16] quantifies the benefits of using ownership to reason about memory access in complex systems [67], and shows how ownership can co-exist with traditional memory reasoning. Linear Dafny’s ownership types are considerably less sophisticated than Rust’s, however. Linear Dafny in turn builds on Cogent’s purely functional support for borrowing [85] and on early work by Wadler [86].

Verus is one of several tools building on Rust. RustBelt [87] focuses on manually verifying unsafe Rust code using Coq. Prusti [64] encodes Rust code into the Viper verification framework [88], which leads Prusti to essentially re-verify Rust’s typechecking, resulting in larger SMT queries. While conceptually similar to Verus, Creusot [65] lacks Verus’s ability to reason about ghost resources. As §4.1 shows, Verus verifies equivalent code faster than either Prusti or Creusot. Aeneas [89] translates Rust code into a pure functional form that the developer then reasons about in a separate proof assistant (currently Lean [32]); this differs from Verus’s intrinsic approach where the developer writes code and proofs directly in Rust. In addition, Prusti, Creusot, and Aeneas do not offer EPR-style automation, nor do they include support for system-specific idioms or reasoning about concurrency.

Prior work on Verus [10] focuses on Verus’s ghost ownership mechanism and formalizes the interactions between spec, proof, and executable code. It also shows how Verus can reason about `unsafe` Rust code using owned ghost variables.

This paper focuses on systems aspects of how Verus:

1. Optimizes SMT performance via context pruning and the careful design of quantifier trigger-selection (§3.1);
2. Provides the proof-free automation (§3.2) of tools like Ivy [7, 8, 18–22] by allowing a developer to opt into the restricted EPR logic on a per-module basis, and then soundly connect such proofs to code written in unrestricted logic;
3. Incorporates simple-to-invoke proof automation for non-linear arithmetic, bit-vectors, and proof-by-computation, both to address systems-verification needs and to keep the main SMT encoding simple and efficient (§3.3);
4. Introduces VerusSync (§3.4), a novel state-transition-based language to describe operations on ghost state embedded in code via Rust ownership types. Unlike other systems [3, 29, 30] (or prior work on Verus [10]), VerusSync does not require the developer to understand or construct low-level resource algebras.

While others have built individual systems using Verus [25–27], we evaluate the impact of Verus’s design choices through an extensive *comparative evaluation* (§4).

6 Conclusion

Verus aims to consolidate the gains made by the system-verification community in a unified tool. By building on a mainstream language (Rust), Verus makes these gains available to a much broader audience of system developers. At the same time, by leveraging Rust and our carefully designed system-oriented features, Verus provides a higher-level starting point for future research in this area. Ultimately, we hope that Verus enables researchers to identify the exciting research challenges that emerge as we scale verification to new heights of system size and complexity.

Acknowledgments

We thank our shepherd, Ronghui Gu, and the OSDI 2024 and SOSP 2024 anonymous reviewers for helpful feedback on the paper. Thanks also to Cheng Huang and Yiheng Tao from Azure Storage for their help with problem definition, discussion, and integration of the persistent log. Thanks to Gerd Zellweger, who helped improve the design of the verified page table, and to Xavier Denis, who helped write the Creusot version of the millibenchmarks.

Matthias Brun was supported by a gift from VMware. Reto Achermann was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC). Hayley LeBlanc was partially supported by donations from Toyota. Work at CMU was supported, in part, by an Amazon Research Award (Fall 2022 CFP), a gift from VMware, the Future Enterprise Security initiative at Carnegie Mellon CyLab (FutureEnterprise@CyLab), NSF grant CCF 2318953, and funding from AFRL and DARPA under Agreement FA8750-24-9-1000. Chanhee Cho was also supported by the Kwanjeong Educational Foundation.

References

- [1] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. Push-button verification of file systems via crash refinement. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November 2016.
- [2] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nickolai Zeldovich. Using Crash Hoare Logic for certifying the FSCQ file system. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, October 2015.
- [3] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying concurrent, crash-safe systems with Perennial. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, October 2019.
- [4] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. Hyperkernel: Push-button verification of an OS kernel. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2017.
- [5] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. Scaling symbolic evaluation for automated verification of systems code with Serval. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2019.
- [6] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. Specification and verification in the field: Applying formal methods to BPF just-in-time compilers in the Linux kernel. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, November 2020.
- [7] Oded Padon, Kenneth L. McMillan, Mooly Sagiv, Aurojit Panda, and Sharon Shoham. Ivy: Safety verification by interactive generalization. In *Proceedings of the ACM Conference on Programming Language Design and Implementation (PLDI)*, 2016.
- [8] Kenneth L. McMillan and Oded Padon. Ivy: A multi-modal verification tool for distributed algorithms. In *Proceedings of the Conference on Computer Aided Verification (CAV)*, 2020.
- [9] Marcelo Taube, Giuliano Losa, Kenneth L. McMillan, Oded Padon, Mooly Sagiv, Sharon Shoham, James R. Wilcox, and Doug Woos. Modularity for decidability of deductive verification with applications to distributed systems. In *Proceedings of the ACM Conference on Programming Language Design and Implementation (PLDI)*, 2018.
- [10] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. In *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, December 2023.
- [11] Ernie Cohen, Markus Dahlweid, Mark Hillebrand, Dirk Leinenbach, Michał Moskal, Thomas Santen, Wolfram Schulte, and Stephan Tobies. VCC: A practical system for verifying concurrent C. In *Proceedings of the Conference on Theorem Proving in Higher Order Logics*, 2009.
- [12] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. Ironclad apps: End-to-end security via automated full-system verification. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, October 2014.
- [13] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, October 2015.
- [14] Jonathan Protzenko, Bryan Parno, Aymeric Fromherz, Chris Hawblitzel, Marina Polubelova, Karthikeyan Bhargavan, Benjamin Beurdouche, Joonwon Choi, Antoine Delignat-Lavaud, Cedric Fournet, Natalia Kulatova, Tahina Ramananandro, Aseem Rastogi, Nikhil Swamy, Christoph Wintersteiger, and Santiago Zanella-Béguelin. EverCrypt: A fast, verified, cross-platform cryptographic provider. In *Proceedings of the IEEE Symposium on Security and Privacy (Oakland)*, May 2020.
- [15] Marina Polubelova, Karthikeyan Bhargavan, Jonathan Protzenko, Benjamin Beurdouche, Aymeric Fromherz, Natalia Kulatova, and Santiago Zanella-Béguelin. HACLxN: Verified generic SIMD crypto (for all your favorite platforms). In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, October 2020.
- [16] Travis Hance, Andrea Lattuada, C. Hawblitzel, Jon Howell, Rob Johnson, and Bryan Parno. Storage systems are distributed systems (so verify them that way!). In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.
- [17] R. Piskac, L. M. de Moura, and N. Björner. Deciding effectively propositional logic using DPLL and substitution sets. *Journal of Automated Reasoning*, 44(4), 2010.
- [18] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. Paxos made EPR: Decidable reasoning about distributed protocols. In *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, October 2017.
- [19] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem A. Sakallah. I4: Incremental inference of inductive invariants for verification of distributed protocols. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2019.
- [20] Travis Hance, Marijn Heule, Ruben Martins, and Bryan Parno. Finding invariants of distributed systems: It's a small (enough) world after all. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, April 2021.
- [21] Jianan Yao, Runzhou Tao, Ronghui Gu, and Jason Nieh. DuoAI: Fast, automated inference of inductive invariants for verifying distributed protocols. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2022.
- [22] Jianan Yao, Runzhou Tao, Ronghui Gu, Jason Nieh, Suman Jana, and Gabriel Ryan. DistAI: Data-Driven automated invariant learning for distributed protocols. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021.
- [23] The Verus Contributors. Verus repository. <https://github.com/verus-lang/verus>.
- [24] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jay Lorch, Oded Padon, and Bryan Parno. Verus SOSP artifact. <https://verus-lang.github.io/paper-sosp24-artifact/>, 2024.
- [25] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: Verifying liveness of cluster management controllers. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2024.
- [26] Xiangdong Chen, Zhaofeng Li, Lukas Mesicek, Vikram Narayanan, and Anton Burtsev. Atmosphere: Towards practical verified kernels in Rust. In *Proceedings of the Workshop on Kernel Isolation, Safety and Verification (KISV)*, 2023.
- [27] Ziqiao Zhou, Weiteng Chen, Chris Hawblitzel, and Weidong Cui. VeriSMo: A verified security module for confidential VMs. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2024.
- [28] Jianan Yao, Ziqiao Zhou, Weiteng Chen, and Weidong Cui. Leveraging large language models for automated proof synthesis in Rust. <https://arxiv.org/abs/2311.03739>, 2023.
- [29] Travis Hance, Yi Zhou, Andrea Lattuada, Reto Achermann, Alex Conway, Ryan Stutsman, Gerd Zellweger, Chris Hawblitzel, Jon Howell, and Bryan Parno. Sharding the state machine: Automated modular reasoning for complex concurrent systems. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2023.
- [30] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, Frans Kaashoek, and Nickolai Zeldovich. Grove: A separation-logic library for verifying distributed systems. In *Proceedings of the Symposium on Operating*

- Systems Principles (SOSP)*, 2023.
- [31] Coq Development Team. The Coq Proof Assistant <https://coq.inria.fr/>.
 - [32] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover. In *Proceedings of the Conference on Automated Deduction (CADE)*, August 2015.
 - [33] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the IEEE Symposium on Logic in Computer Science*, 2002.
 - [34] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proceedings of the Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*, 2010.
 - [35] Nikhil Swamy, Cătălin Hritcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, Jean-Karim Zinzindohoué, and Santiago Zanella-Béguélin. Dependent types and multi-monadic effects in F*. In *Proceedings of the ACM Conference on Principles of Programming Languages (POPL)*, 2016.
 - [36] Nicholas D. Matsakis and Felix S. Klock. The Rust language. *Ada Lett.*, 34(3):103–104, October 2014.
 - [37] Steve Klabnik and Carol Nichols. *The Rust Programming Language*. No Starch Press, USA, 2018.
 - [38] Liam Proven. Linux 6.1: Rust to hit mainline kernel. https://www.theregister.com/2022/10/05/rust_kernel_pull_request_pulled/, October 2022.
 - [39] Shane Miller and Carl Lerche. Sustainability with Rust. <https://aws.amazon.com/blogs/opensource/sustainability-with-rust/>, February 2022.
 - [40] Google. Announcing KataOS and Sparrow. <https://opensource.googleblog.com/2022/10/announcing-kataos-and-sparrow.html>, October 2022.
 - [41] Yechan Bae, Youngsuk Kim, Ammar Askar, Jungwon Lim, and Taesoo Kim. RUDRA: Finding memory safety bugs in Rust at the ecosystem scale. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, October 2021.
 - [42] Jacob R. Lorch, Yixuan Chen, Manos Kapritsos, Haojun Ma, Bryan Parno, Shaz Qadeer, Upamanyu Sharma, James R. Wilcox, and Xueyuan Zhao. Armada: Automated verification of concurrent code with sound semantic extensibility. *ACM Transactions on Programming Languages and Systems*, 44(2), June 2022.
 - [43] Andrew Ferraiuolo, Andrew Baumann, Chris Hawblitzel, and Bryan Parno. Komodo: Using verification to disentangle secure-enclave hardware from software. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, October 2017.
 - [44] Yi Zhou, Jay Bosamiya, Yoshiaki Takashima, Jessica Li, Marijn Heule, and Bryan Parno. Mariposa: Measuring SMT instability in automated program verification. In *Proceedings of the Formal Methods in Computer-Aided Design (FMCAD) Conference*, October 2023.
 - [45] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
 - [46] L. M. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proceedings of the Conference on Tools and Algorithms for the Construction and Analysis of Systems*, 2008.
 - [47] Robert Floyd. Assigning meanings to programs. In *Proceedings of the Symposia in Applied Mathematics*, 1967.
 - [48] Tony Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12, 1969.
 - [49] Michał Moskal. Programming with triggers. In *Proceedings of the Workshop on Satisfiability Modulo Theories*, 2009.
 - [50] Oded Padon, Jochen Hoenicke, Giuliano Losa, Andreas Podelski, Mooly Sagiv, and Sharon Shoham. Reducing liveness to safety in first-order logic. *PACMPL*, 2(POPL):26:1–26:33, 2018.
 - [51] Jason R. Koenig, Oded Padon, Sharon Shoham, and Alex Aiken. Inferring invariants with quantifier alternations: Taming the search space explosion. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 13243, 2022.
 - [52] Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang. Certified verification of algebraic properties on low-level mathematical constructs in cryptographic programs. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, 2017.
 - [53] Yi Zhou, Sydney Gibson, Sarah Cai, Menucha Winchell, and Bryan Parno. Galápagos: Developing verified low-level cryptography on heterogeneous hardware. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, November 2023.
 - [54] Jacob R. Lorch, Yixuan Chen, Manos Kapritsos, Haojun Ma, Bryan Parno, Shaz Qadeer, Upamanyu Sharma, James R. Wilcox, and Xueyuan Zhao. Armada code repository. <https://github.com/microsoft/Armada/blob/ee799110f9aecc3deab31b94521bdbcd27f58363/Test/qbss/bv.dfy#L54>, December 2023.
 - [55] Martijn Oostdijk and Herman Geuvers. Proof by computation in the Coq system. *Theoretical Computer Science*, 272(1–2), February 2002.
 - [56] Guido Martínez, Danel Ahman, Victor Dumitrescu, Nick Giannarakis, Chris Hawblitzel, Catalin Hritcu, Monal Narasimhamurthy, Zoe Paraskevopoulou, Clément Pit-Claudel, Jonathan Protzenko, Tahina Ramananandro, Aseem Rastogi, and Nikhil Swamy. Meta-F*: Proof automation with SMT, tactics, and metaprograms. In *Proceedings of the European Symposium on Programming (ESOP)*, April 2019.
 - [57] Ralf Jung, R. Krebbers, Jacques-Henri Jourdan, A. Bizjak, L. Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28, 2018.
 - [58] Peter W. O’Hearn. Resources, concurrency, and local reasoning. *Theoretical Computer Science*, 375(1–3), April 2007.
 - [59] Barry Bond, Chris Hawblitzel, Manos Kapritsos, K. Rustan M. Leino, Jacob R. Lorch, Bryan Parno, Ashay Rane, Srinath Setty, and Laure Thompson. Vale: Verifying high-performance cryptographic assembly code. In *Proceedings of the USENIX Security Symposium*, August 2017.
 - [60] Jonathan Protzenko, Jean-Karim Zinzindohoué, Aseem Rastogi, Tahina Ramananandro, Peng Wang, Santiago Zanella-Béguélin, Antoine Delignat-Lavaud, Catalin Hritcu, Karthikeyan Bhargavan, Cédric Fournet, and Nikhil Swamy. Verified low-level programming embedded in F*. *PACMPL*, 1(ICFP), September 2017.
 - [61] Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Jiangyang Pan, Jonathan Protzenko, Aseem Rastogi, Nikhil Swamy, Santiago Zanella-Béguélin, and Jean Karim Zinzindohoué. Implementing and proving the TLS 1.3 record layer. In *Proceedings of the IEEE Symposium on Security and Privacy*, May 2017.
 - [62] Antoine Delignat-Lavaud, Cédric Fournet, Bryan Parno, Jonathan Protzenko, Tahina Ramananandro, Jay Bosamiya, Joseph Lallemand, Itsaka Rakotonirina, and Yi Zhou. A security model and fully verified implementation for the IETF QUIC record layer. In *Proceedings of the IEEE Symposium on Security and Privacy*, May 2021.
 - [63] Jonathan Protzenko, Benjamin Beurdouche, Denis Merigoux, and Karthikeyan Bhargavan. Formally verified cryptographic web applications in WebAssembly. In *Proceedings of the IEEE Symposium on Security and Privacy*, May 2019.
 - [64] Fabian Wolff, Aurel Bily, Christoph Matheja, Peter Müller, and Alexander J. Summers. Modular specification and verification of closures in Rust. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA), October 2021.
 - [65] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. Creusot: A foundry for the deductive verification of Rust programs. In *Proceedings of the International Conference on Formal Engineering Methods*, October 2022.
 - [66] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Haojun Ma, Bryan Parno, Michael L. Roberts, Srinath Setty, and Brian Zill. IronFleet code. <https://research.microsoft.com/projects/ironclad/>,

- 2015.
- [67] Jialin Li, Andrea Lattuada, Yi Zhou, Jonathan Cameron, Jon Howell, Bryan Parno, and Chris Hawblitzel. Linear types for large-scale systems verification. In *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, November 2022.
 - [68] Ankit Bhardwaj, Chinmay Kulkarni, Reto Achermann, Irina Calciu, Sanidhya Kashyap, Ryan Stutsman, Amy Tai, and Gerd Zellweger. NrOS: Effective replication and sharing in an operating system. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2021.
 - [69] Travis Hance, Yi Zhou, Andrea Lattuada, Reto Achermann, Alex Conway, Ryan Stutsman, Gerd Zellweger, Chris Hawblitzel, Jon Howell, and Bryan Parno. IronSync OSDI 2023 artifact. <https://github.com/secure-foundations/ironsync-osdi2023>, 2023.
 - [70] Daan Leijen, Ben Zorn, and Leonardo de Moura. Mimalloc: Free list sharding in action. Technical Report MSR-TR-2019-18, Microsoft, June 2019.
 - [71] Daan Leijen. Mimalloc-bench. <https://github.com/daanx/mimalloc-bench>, 2023.
 - [72] Intel. Intel Optane persistent memory. <https://www.intel.com/content/www/us/en/products/docs/memory-storage/optane-persistent-memory/overview.html>.
 - [73] Jian Xu, Lu Zhang, Amirsaman Memaripour, Akshatha Gangadharaiah, Amit Borase, Tamires Brito Da Silva, Steven Swanson, and Andy Rudoff. NOVA-Fortis: A fault-tolerant non-volatile main memory file system. In *Proceedings of the ACM Symposium on Operating Systems Principles, (SOSP)*, 2017.
 - [74] Linux Kernel Developers. Direct Access for files. <https://www.kernel.org/doc/Documentation/filesystems/dax.txt>.
 - [75] PMDK Developers. libpmemlog. <https://pmem.io/pmdk/manpages/linux/v1.3/libpmemlog.3/>.
 - [76] PMDK Developers. PMDK. <https://pmem.io/pmdk/>.
 - [77] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. CakeML: A verified implementation of ML. In *Proceedings of the ACM Symposium on Principles of Programming Languages (POPL)*, January 2014.
 - [78] Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski, and Gernot Heiser. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems*, 32(1), 2014.
 - [79] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM (CACM)*, 52(7):107–115, 2009.
 - [80] James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings of the ACM Conference on Programming Language Design and Implementation (PLDI)*, June 2015.
 - [81] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. CertiKOS: An extensible architecture for building certified concurrent OS kernels. In *Proceedings of the USENIX Conference on Operating Systems Design and Implementation (OSDI)*, 2016.
 - [82] Xupeng Li, Xuheng Li, Wei Qiang, Ronghui Gu, and Jason Nieh. Spoq: Scaling machine-checkable systems verification in Coq. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, July 2023.
 - [83] Tobias Nipkow, Lawrence Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.
 - [84] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Michael Norrish, Rafal Kolanski, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an OS kernel. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2009.
 - [85] Sidney Amani, Alex Hixon, Zilin Chen, Christine Rizkallah, Peter Chubb, Liam O'Connor, Joel Beeren, Yutaka Nagashima, Japheth Lim, Thomas Sewell, Joseph Tuong, Gabriele Keller, Toby Murray, Gerwin Klein, and Gernot Heiser. Cogent: Verifying high-assurance file system implementations. In *Proceedings of the ACM Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2016.
 - [86] Philip Wadler. Linear types can change the world! In *Proceedings of the IFIP TC 2 Working Conference on Programming Concepts and Methods*, 1990.
 - [87] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL), January 2018.
 - [88] Peter Müller, Malte Schwerhoff, and Alexander J. Summers. Viper: A verification infrastructure for permission-based reasoning. In *Proceedings of the Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, 2016.
 - [89] Son Ho and Jonathan Protzenko. Aeneas: Rust verification by functional translation. *Proceedings of the ACM on Programming Languages*, 6(ICFP):711–741, 2022.